实验二：搜索算法实验报告

GitHub 仓库：https://github.com/shixuhui-ops/AI-Search-Experiment-2

实验二：搜索算法实验

GitHub 仓库：

https://github.com/shixuhui-ops/AI-Search-Experiment-2

一、实验目标

本实验实现并测试 A* 搜索、加权地形 A* 搜索、Minimax、Alpha-Beta 剪枝，以及结合 A* 和博弈搜索的宝藏分配问题。

实验重点是理解搜索状态、代价函数、启发函数、优先队列、极大极小递归和 Alpha-Beta
剪枝条件，并通过实验数据分析不同策略的性能差异。

二、算法设计

1. 迷宫 A*

状态为网格中的坐标 `(row, col)`。

□ `g(n)`：从起点到当前节点的实际移动代价。

□ `h(n)`：当前节点到终点的曼哈顿距离。

□ `f(n) = g(n) + h(n)`。

□ 每次从优先队列中取出 `f(n)` 最小的节点。

□ 障碍物 `X` 不允许进入。

□ 每次移动的代价为 1。

对于四方向移动，曼哈顿距离不会高估实际剩余步数，因此是可采纳启发函数，可以保证 A* 找到最短路径。

2. 加权地形 A*

加权地图中，进入相邻格子的代价等于目标格子的地形值，进入终点 `E` 的代价为 0。

算法仍使用优先队列进行搜索，并将累计地形代价作为 `g(n)`。无解时返回 `-1`。

3. Minimax

根节点为 MAX 玩家，下一层为 MIN 玩家，双方交替进行。

□ MAX 节点选择子节点的最大值。

□ MIN 节点选择子节点的最小值。

□ 叶子节点直接返回对应分数。

4. Alpha-Beta 剪枝

Alpha-Beta 在 Minimax 基础上维护两个边界：

□ `alpha`：MAX 当前已经找到的最好值。

□ `beta`：MIN 当前已经找到的最好值。

当 `alpha >= beta` 时，剩余分支不可能改变祖先节点的最终决策，因此可以直接剪枝。

三、题目一：迷宫寻路结果

测试地图中，起点为 `(0, 0)`，终点为 `(3, 4)`。

最短路径为：

[(0, 0), (0, 1), (0, 2), (1, 2),

(1, 3), (1, 4), (2, 4), (3, 4)]

路径包含 8 个坐标，实际移动 7 步。

使用 `h=0` 时扩展 16 个节点，使用曼哈顿距离时扩展 14 个节点。两种方法得到相同的最优路径。

四、题目二：加权地形结果

加权地图中，进入不同地形的代价不同，进入终点 `E` 的代价为 0。

算法计算得到从起点到终点的最小总代价：

7

该结果说明加权搜索不能只比较路径长度，还需要比较经过地形的累计代价。某些路径虽然经过的格子更多，但由于避开了高代价地形，总代价反而更低。

五、题目三：Minimax 与 Alpha-Beta 结果

在题目给出的博弈树上：

Minimax 结果：5

Alpha-Beta 结果：5

两种算法得到相同的最优值，说明 Alpha-Beta 剪枝不会改变 Minimax 的正确结果，只会减少不必要的搜索。

在相同测试树上：

Minimax 访问节点数：15

Alpha-Beta 访问节点数：14

剪枝节点数：1

六、题目四：宝藏分配博弈

点的顺序为：

[A, B, T1, T2, T3, T4]

距离矩阵为：

[0, 12, 10, 4, 8, 8]

[12, 0, 8, 8, 4, 10]

[10, 8, 0, 6, 6, 12]

[4, 8, 6, 0, 6, 6]

[8, 4, 6, 6, 0, 6]

[8, 10, 12, 6, 6, 0]

宝藏价值为：

T1 = 9

T2 = 7

T3 = 8

T4 = 7

收益计算方式：

Gain = 宝藏价值 - 移动代价

U = Score_A - Score_B

A 玩家选择使 `U` 最大的策略，B 玩家选择使 `U` 最小的策略。

搜索深度为 2 时：

最优首步：T2

最优收益差：-1

搜索深度为 4 时：

最优首步：T3

最优收益差：3

Minimax 和 Alpha-Beta 在两个搜索深度下都得到相同结果。

七、性能分析

1. A* 启发函数对比

在相同迷宫上进行测试：

h=0：扩展 16 个节点

曼哈顿距离：扩展 14 个节点

节点数减少：

2 个

减少比例：

12.5%

两种方法的单次运行时间均为亚毫秒级，具体时间会受到机器调度和计时误差影响。因此，本实验主要使用扩展节点数量评价启发函数效果。

2. Alpha-Beta 剪枝对比

在相同测试树上：

Minimax：访问 15 个节点

Alpha-Beta：访问 14 个节点

Alpha-Beta 减少访问 1 个节点，同时保持最优值为 5。

3. 大规模博弈树

在深度为 4、每个节点有 3 个子节点的测试树上：

原始访问顺序：访问 73 个节点

反转访问顺序：访问 45 个节点

两种顺序的最优值：141

反转访问顺序后少访问 28 个节点，说明 Alpha-Beta
的剪枝效果与节点访问顺序密切相关。合理的节点排序可以显著提高搜索效率。

八、优化策略

1. 启发函数优化

普通迷宫使用曼哈顿距离作为启发函数，在保证最优性的同时减少节点扩展。

2. 距离矩阵对称性优化

地图移动代价具有对称性，因此距离矩阵只计算上三角部分，再将结果镜像填充到下三角，避免重复执行 A*。

3. Minimax 记忆化搜索

宝藏 Minimax 使用 `lru_cache` 缓存已经计算过的状态，避免重复计算相同的玩家位置、剩余宝藏、累计得分和当前回合。

Alpha-Beta 没有直接缓存剪枝结果，因为剪枝返回值可能受到当前 `alpha` 和 `beta` 边界影响。

九、边界情况测试

当迷宫中的起点和终点之间不存在可行路径时：

astar_path(start, goal, grid)

返回：

[]

当加权地图中的起点和终点之间不存在可行路径时：

weighted_astar_cost(start, goal, grid)

返回：

-1

测试结果表明，程序能够正确处理无解输入，不会进入死循环，也不会返回错误路径。

十、自动化测试

使用 Python unittest 进行自动化测试，共包含 9 个测试用例：

☐ 迷宫 A* 路径搜索

☐ 迷宫无解情况

☐ 加权地形 A*

☐ 加权地图无解情况

☐ Minimax

☐ Alpha-Beta

☐ A* 性能统计

□ 二叉树反转

□ Alpha-Beta 访问顺序

测试结果：

Ran 9 tests

OK

所有测试均通过。

十一、测试覆盖率

使用 coverage 统计测试覆盖率：

python -m coverage run -m unittest test_solution.py

python -m coverage report -m

当前结果为：

solution.py 覆盖率约为 87%

项目整体覆盖率约为 88%

剩余未覆盖部分主要是演示入口、异常分支和少量不影响核心算法正确性的代码。

十二、核心实验结果对比

实验项目 方法或参数 结果 性能数据

迷宫寻路 A*，h=0 路径长度 8 扩展 16 个节点

迷宫寻路 A*，曼哈顿启发式 路径长度 8 扩展 14 个节点

加权地形 加权 A* 最小代价 7 避开高代价地形

博弈树 Minimax 最优值 5 访问 15 个节点

博弈树 Alpha-Beta 最优值 5 访问 14 个节点

宝藏博弈 H=2 T2，收益差 -1 两种算法一致

宝藏博弈 H=4 T3，收益差 3 两种算法一致

从结果可以看出，合理的启发函数可以减少 A* 的搜索节点，Alpha-Beta 可以减少博弈树搜索节点，并且不会改变最终决策结果。

十三、Alpha-Beta 搜索过程示意图

MAX

/ \

MIN MIN

/ \ / \

MAX MAX MAX MAX

/ \ / \ / \ / \

3 5 6 9 1 2 4 7

搜索从根节点开始，MAX 层选择较大值，MIN 层选择较小值。当当前 alpha 值不小于 beta 值时，后续分支不再可能改变祖先节点的决策，因此可以剪枝。

十四、可复现性与工程化

项目提供：

□ `requirements.txt`：项目依赖。

□ `verify.ps1`：一键验证脚本。

□ `benchmark.py`：性能基准测试。

□ `.github/workflows/tests.yml`：GitHub Actions 自动测试。

安装依赖：

python -m pip install -r requirements.txt

执行一键验证：

powershell -ExecutionPolicy Bypass -File .\verify.ps1

GitHub Actions 会自动执行：

□ Python 语法检查

□ 9 个单元测试

□ coverage 覆盖率统计

□ 覆盖率报告保存

十五、实验总结

本实验完成了迷宫 A*、加权地形 A*、Minimax、Alpha-Beta 剪枝和宝藏分配博弈的实现。

实验结果与题目给出的预期结果一致。实验表明：

1. 曼哈顿启发函数可以减少 A* 的搜索节点。
2. 加权 A* 能够根据地形代价选择总代价更低的路径。
3. Alpha-Beta 可以在不改变 Minimax 结果的情况下减少搜索节点。
4. 节点访问顺序会明显影响 Alpha-Beta 的剪枝效果。
5. 记忆化搜索和距离矩阵对称性可以减少重复计算。
6. 自动化测试和 GitHub Actions 提高了代码的可复现性和可靠性。